CẨM NANG ÔN THI & THỰC CHIẾN BẢO VỆ ĐỒ ÁN PROPNEST (100% BACKEND)

Tài liệu này được thiết kế chuyên biệt cho cấu trúc bài thi của bạn:

- **Phần A (45 phút — Live Coding Tầng 1)**: Mở rộng/sửa trực tiếp chức năng có sẵn trên mã nguồn dự án (giống hệt format **Đề số 7** trong ảnh bạn gửi: sửa API tìm kiếm/lọc, thêm điều kiện phân quyền, thêm trường/validation, sửa logic nghiệp vụ).
- **Phần B (10 phút — Xác nhận kỹ thuật nâng cao Tầng 3)**: Chỉ đúng file, đúng dòng code, giải thích cơ chế hoạt động của **Saga Orchestration + Hoàn tiền tự động, Transactional Outbox, Inbox Deduplication, Idempotency-Key (SHA-256), Optimistic Concurrency (`RowVersion`), History `DeltaEngine`, Quartz.NET, Serilog**.

PHẦN 1: BẢN ĐỒ ĐỊNH VỊ MÃ NGUỒN (HỎI ĐẾN ĐÂU — MỞ ĐÚNG FILE ĐÓ TRONG 3 GIÂY)

Khi giảng viên hỏi bất kỳ tính năng nào trong báo cáo, bạn dùng bảng này để mở đúng file ngay lập tức:

Chức năng / Kỹ thuật trong Báo cáo	Chương tham chiếu	File C# thực tế trong Solution	Dòng code trọng tâm
Khởi tạo DI, JWT, Policy, Seed Roles/Admin, Pipeline	Chương 3, 4, 7		`L17-L92` (DI/Auth), `L118-L128` (Pipeline), `L133-L194` (Seed)
Gắn mã truy vết `X-Correlation-Id` vào Request/Response/Log	Chương 4, 6, 9		`L11-L26` (`InvokeAsync` , `BeginScope`)
Bắt lỗi toàn cục chuẩn RFC 7807 `ProblemDetails`	Chương 4, 6	&	`L15-L64` (Middleware), `L8-L37` (`HandleException`)
Đăng ký nguyên tử (User + Role + Wallet) & Login Lockout 5 lần	Chương 7, Lab 05		`L30-L60` (`BeginTransactionAsync`), `L75-L86` (Lockout `423`)
Sinh JWT Token (Claims `sub` , `email` , `role` , HMAC-SHA256)	Chương 7, Lab 05		`L14-L53` (`GenerateAccessToken`)
Lấy `UserId` từ JWT & Chống IDOR/BOLA (`EnsureOwner`)	Chương 7, Lab 05	&	`L7-L23` (`EnsureOwner` , `EnsureCanViewHistory`)
Thực thể `Listing` (8 trạng thái, Domain Guard Clauses)	Chương 5, 11, Lab 02		`L61-L165` (`UpdateContent` , `Submit` , `Approve` , `Reject` , `Hide` , `Expire`)
API CRUD Tin đăng, Tìm kiếm, `RowVersion` , Mua gói	Chương 6, 11, Lab 04		`L59-L101` (`Update` + Base64 `RowVersion`), `L136-L161` (`PurchasePackage`)
Logic nghiệp vụ Tin đăng + Ghi History + Ghi Outbox	Chương 6, 11, Lab 04		`L69-L104` (`UpdateAsync`), `L243-L312` (`ApproveAsync` / `RejectAsync`)
Truy vấn EF Core (`AsNoTracking` , Projection, `SetOriginalRowVersion`)	Chương 5, 9, Lab 04		`L104-L112` (`RowVersion`), `L114-L164` (`GetPublicListingsAsync`)
Thuật toán bóc tách vi sai `History Delta Engine`	Chương 9, 11, Lab 04		`L11-L39` (`CreateDeltas` ,

Chức năng / Kỹ thuật trong Báo cáo	Chương tham chiếu	File C# thực tế trong Solution	Dòng code trọng tâm
			`EqualityComparer<T>.Default`)
Thực thể `Wallet` (Ví 2 ngăn: trừ `PromoBalance` trước, `MainBalance` sau)	Chương 5, 11, Lab 02		`L34-L50` (`Charge` , `Refund` , `CreditMain`)
Khởi tạo mua gói + Kiểm tra `Idempotency-Key` (SHA-256) + Ghi Outbox	Chương 8, 11, Lab 06		`L11-L65` (`StartPurchaseAsync` , `ComputeRequestHash`)
Worker quét bảng `OutboxMessages` mỗi 5 giây đẩy lên RabbitMQ	Chương 8, Lab 06	&	`L26-L56` (vòng lặp 5s), `L10-L35` (`Take(50)` , `MarkProcessed`)
Bộ lọc chống xử lý trùng message `InboxDeduplicationFilter<T>`	Chương 8, Lab 06		`L22-L88` (kiểm tra `(ConsumerName, MessageId)` trong `InboxMessages`)
Cỗ máy trạng thái `PurchasePackageSaga` (MassTransit State Machine)	Chương 8, 11, Lab 06		`L36-L150` (`Initially` , `During(DeductingWallet)` , `During(UpgradingListing)` , `During(Compensating)`)
3 Consumers của Saga (Trừ ví, Nâng cấp tin, Hoàn tiền tự động)	Chương 8, 11, Lab 06	, ,	Phương thức `Consume(...)` ở cả 3 file
Quartz.NET Job tự động hạ tin hết hạn (`ListingExpirationJob`)	Chương 9	&	Cron `0 0/5 * * * ?` trong

PHẦN 2: GIẢI PHẪU CHI TIẾT 5 LUỒNG KỸ THUẬT KHÓ NHẤT (ĐỌC ĐỂ HIỂU BẢN CHẤT 100%)

Luồng 1: Đường ống Middleware Pipeline & Xử lý lỗi RFC 7807

Nằm ở đâu?

```
csharp
app.UseMiddleware<ExceptionHandlingMiddleware>(); // (1) Đứng ngoài cùng để bọc try-catch cho toàn bộ các tầng bên trong
app.UseMiddleware<CorrelationIdMiddleware>();     // (2) Sinh/đọc X-Correlation-Id và mở LogScope trước khi vào Auth/Controller
app.UseHttpsRedirection();                        // (3) Chuyển hướng HTTP -> HTTPS
app.UseAuthentication();                          // (4) Giải mã JWT Bearer Token từ header Authorization -> gán vào HttpContext.User
app.UseAuthorization();                           // (5) Kiểm tra [Authorize(Roles=...)] hoặc [Authorize(Policy=...)] dựa trên HttpContext.User
app.MapControllers();                             // (6) Điều hướng vào Controller Action
```

> IMPORTANT
>
> **Câu hỏi kinh điển của Giảng viên**:
>
> 1. "*Nếu anh/chị đổi chỗ `UseAuthorization()` lên trước `UseAuthentication()` thì chuyện gì xảy ra?*"
>   - **Trả lời**: Khi request đi qua `UseAuthorization()`, token JWT chưa được giải mã nên `HttpContext.User` vẫn là người dùng ẩn danh (chưa xác thực). Do đó mọi API có gắn `[Authorize]` đều bị từ chối với lỗi **`401 Unauthorized`** dù Client gửi kèm Token hợp lệ!
> 2. "*Tại sao trong `ListingsController` có một số hàm dùng `try { ... } catch (Exception ex) { return this.HandleException(ex); }`, nhưng hệ thống vẫn có `ExceptionHandlingMiddleware`?*"
>   - **Trả lời**: Các action như `Update`, `Delete`, `Submit`, `Hide` trong bắt lỗi cục bộ qua extension để trả về `ProblemDetails` ngay tại Controller (trong đó `InvalidOperationException` -> `409 Conflict`). Còn các Controller không đặt `try-catch` (như `ModerationController`, `ListingsController.Create`, `PurchasePackage`) khi ném ngoại lệ sẽ rơi ra ngoài cho bắt tự động.

Luồng 2: Xác thực (Authentication), Khóa tài khoản (Lockout) & Phân quyền 3 lớp (Authorization)

2.1. Đăng ký tài khoản nguyên tử (Atomic Registration)

- **File**:
- **Tại sao phải dùng `await using var transaction = await dbContext.Database.BeginTransactionAsync(cancellationToken);`?** Vì khi 1 Seller đăng ký mới,

hệ thống thực hiện **3 lệnh ghi xuống 3 bảng khác nhau**:

1. `userManager.CreateAsync(user, request.Password)` -> Ghi bảng `AspNetUsers` (mật khẩu tự động được Identity băm bằng **PBKDF2 HMAC-SHA512** kèm Salt ngẫu nhiên).
2. `userManager.AddToRoleAsync(user, "Seller")` -> Ghi bảng `AspNetUserRoles`.
3. `walletService.CreateWalletAsync(user.Id)` -> Ghi bảng `Wallets` (tạo ví số dư `0`). Nếu bước (1) và (2) thành công nhưng bước (3) tạo ví bị lỗi mà không có Transaction, DB sẽ sinh ra tài khoản "mồ côi" không có ví tiền. Nhờ `BeginTransactionAsync` + `CommitAsync` (và `RollbackAsync` nếu bất kỳ bước nào lỗi), cả 3 bước **hoặc cùng thành công, hoặc cùng hủy bỏ (ACID Atomicity)**.

2.2. Đăng nhập, Cấp JWT & Chống vét cạn mật khẩu (Lockout)

- **Cấu hình Lockout**: Trong :
  - `MaxFailedAccessAttempts = 5` (tối đa 5 lần sai).
  - `DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15)` (khóa 15 phút).
- **Luồng chạy trong** :
  1. Tìm user theo email (`FindByEmailAsync`). Không có -> `401 Unauthorized`.
  2. Kiểm tra `await userManager.IsLockedOutAsync(user)` -> Nếu đang bị khóa, trả về ngay `423 Locked` ("*Tài khoản tạm thời bị khóa do nhập sai mật khẩu quá 5 lần...*").
  3. Kiểm tra mật khẩu `await userManager.CheckPasswordAsync(user, request.Password)`:
     - Nếu **SAI**: gọi `await userManager.AccessFailedAsync(user)` (tăng cột `AccessFailedCount` trong DB lên 1; khi chạm mốc 5 thì Identity tự gán `LockoutEnd = UtcNow + 15 phút`) rồi trả về `401`.
     - Nếu **ĐÚNG**: gọi `await userManager.ResetAccessFailedCountAsync(user)` (reset bộ đếm lỗi về 0), lấy danh sách Roles và gọi .

2.3. Phân quyền 3 lớp (3-Layer Authorization)

1. **Lớp 1 — Role-based (RBAC)**: Dùng `[Authorize(Roles = "Seller")]` trên các API chỉ dành cho người bán ().
2. **Lớp 2 — Policy-based**: Khai báo ở :

```
csharp
options.AddPolicy("CanModerateListings", policy => policy.RequireRole("Moderator",
"Admin"));
```

Gắn lên đầu : `[Authorize(Policy = "CanModerateListings")]`.

- Không gửi token -> `401 Unauthorized`.

- Gửi token của `Seller` -> `403 Forbidden`.
- Gửi token của `Moderator` hoặc `Admin` -> `200 OK`.

3. **Lớp 3 — Resource Ownership (Chống lỗ hổng IDOR / BOLA)**:
   - Dù Seller A đã có Role `Seller`, Seller A không được phép sửa/ẩn/xóa tin đăng của Seller B.
   - đọc claim `ClaimTypes.NameIdentifier` (được ánh xạ từ claim `sub` trong JWT) ra thành `long? UserId`.
   - Hàm kiểm tra `currentUser.UserId.Value != ownerUserId` -> nếu khác nhau thì ném `UnauthorizedAccessException` -> trả về **`403 Forbidden`**.

Luồng 3: Quản lý Tin đăng, Khóa lạc quan (`RowVersion`) & `History Delta Engine`

3.1. Vòng đời trạng thái của thực thể `Listing` (Rich Domain Model)

Trong , mọi thuộc tính đều là `private set;` (không ai ở ngoài gán bừa `listing.Status = ListingStatus.Published` được). Muốn đổi trạng thái phải gọi hàm nghiệp vụ:

- `new Listing(...)` -> Khởi tạo ở trạng thái **`Draft`**, quyết định **`NotRequested`**, gói mặc định **`Standard`**.
- `listing.SubmitForModeration()` (`L85-L96`): Chỉ cho phép khi đang ở `Draft`, `Rejected`, hoặc `Hidden`. Chuyển sang **`PendingModeration`**. Nếu đang `Published` hoặc `PendingModeration` mà gọi hàm này sẽ ném `InvalidOperationException`!
- `listing.Approve(startDate, endDate)` (`L98-L114`): Chỉ cho phép khi đang `PendingModeration`. Chuyển sang **`Published`**, gán `StartDate` và `EndDate = StartDate + package.DurationDays`.
- `listing.Reject()` (`L116-L125`): Chỉ cho phép khi đang `PendingModeration`. Chuyển sang **`Rejected`**.
- `listing.UpdateContent(...)` (`L61-L83`): Cập nhật nội dung tin. **Điểm ăn tiền**: Nếu tin đang ở trạng thái `Published` hoặc `Hidden` mà Seller sửa nội dung, hàm này **tự động ép trạng thái về lại** **`PendingModeration`** (`L76-L80`) để bắt buộc kiểm duyệt lại, chống trò "treo đầu dê bán thịt chó" (duyệt xong rồi sửa nội dung bậy bạ).
- `listing.Delete()` (`L153-L164`): Xóa mềm chuyển sang **`Deleted`** (trong chỉ cho phép xóa khi đang ở `Draft`).

3.2. Kiểm soát đồng thời lạc quan (Optimistic Concurrency Control — `RowVersion`)

- **Bối cảnh**: 2 tab trình duyệt cùng mở sửa tin đăng số 10 (đang có `RowVersion` = `AAAAAAAAB9E=`).
- **Cách hoạt động trong code**:

1. Ở : `builder.Property(x => x.RowVersion).IsRowVersion();` -> SQL Server tạo cột kiểu `rowversion` (8 bytes tự tăng mỗi khi dòng đó bị `UPDATE`).
2. Khi Client gửi `PUT /api/v1/listings/10` kèm chuỗi Base64 `rowVersion`, giải mã `Convert.FromBase64String(request.RowVersion)` thành `byte[]`.
3. Trong , hệ thống gọi :

```
csharp
dbContext.Entry(listing).Property(x => x.RowVersion).OriginalValue = rowVersion;
```

4. Khi gọi `await unitOfWork.SaveChangesAsync()`, EF Core sinh câu lệnh SQL:

```
sql
UPDATE [Listings] SET [Price] = @p0, ...
WHERE [ListingId] = @p1 AND [RowVersion] = @p2;
```

5. Nếu Tab A đã lưu trước, SQL Server đã đổi `RowVersion` trong bảng lên `AAAAAAAAB9I=`. Khi Tab B gửi `RowVersion` cũ (`AAAAAAAAB9E=`), điều kiện `WHERE` không khớp dòng nào (`0 rows affected`) -> EF Core ném `DbUpdateConcurrencyException` -> hệ thống bắt lại và trả về `409 Conflict`.

3.3. Thuật toán bóc tách vi sai `History Delta Engine`

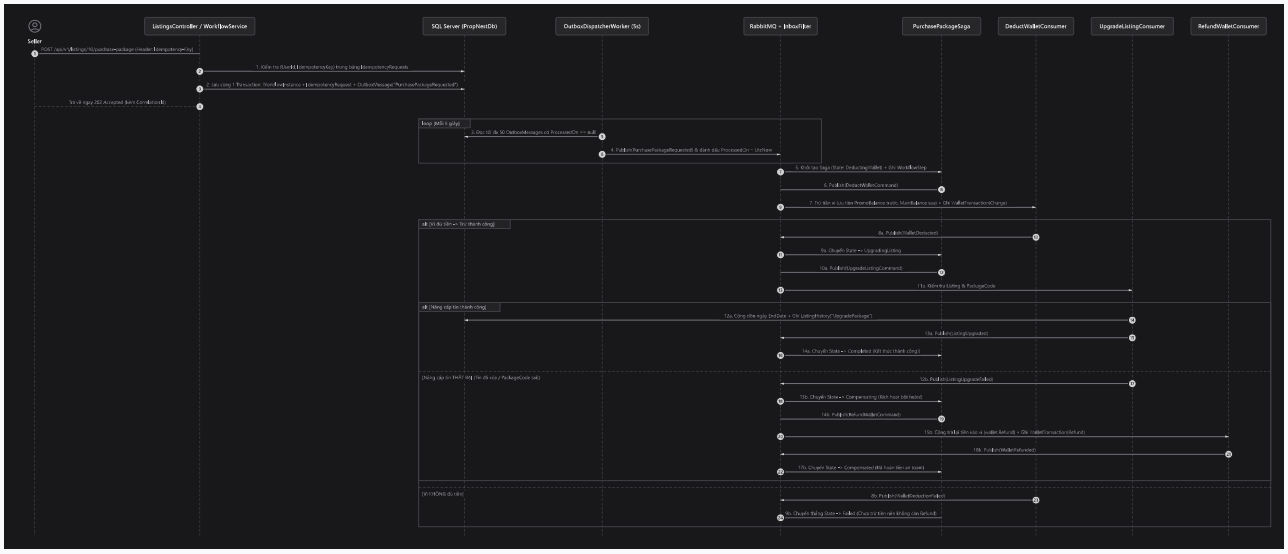
- **File**: &
- **Cách hoạt động**:
    1. Khi **tạo mới tin** (`CreateAsync`, `L30-L34`): Hệ thống lưu 1 bản chụp toàn phần (**Full Snapshot**) bằng `ListingHistory.CreateSnapshot(..., JsonSerializer.Serialize(ToDto(listing)))` vào cột `SnapshotJson` (`DeltasJson = null`).
    2. Khi **cập nhật tin** (`UpdateAsync`, `L72-L95`):
        - Trước khi sửa: chụp trạng thái cũ `var before = ToDto(listing);` (`L72`).
        - Sau khi sửa: chụp trạng thái mới `var after = ToDto(listing);` (`L88`).
        - Gọi `var deltas = DeltaEngine.CreateDeltas(before, after);` (`L89`).
        - Trong :

```
csharp
private static void AddDelta<T>(List<ListingDelta> deltas, string
fieldName, T oldValue, T newValue)
{
    if (!EqualityComparer<T>.Default.Equals(oldValue, newValue))
    {
        deltas.Add(new ListingDelta(fieldName, oldValue, newValue));
    }
}
```

- **Tại sao dùng `EqualityComparer<T>.Default.Equals` mà không dùng `==` hay `.Equals()`?** Vì: (1) An toàn tuyệt đối với `null` (ví dụ `Description` từ `null` đổi sang `"Có nội thất"` hoặc ngược lại không bao giờ bị `NullReferenceException`); (2) Không gây **Boxing** cho các kiểu `struct` như `decimal Price`, `decimal Area`.
- Chỉ những trường thực sự thay đổi mới được đưa vào mảng JSON lưu ở cột `DeltasJson` (`SnapshotJson = null`).

Luồng 4: Giao dịch Phân tán Mua gói VIP — `Idempotency-Key` + `Outbox` + `Inbox` + `PurchasePackageSaga`

Đây là luồng quan trọng nhất ở **Phần B (Tầng 3)** mà bạn nói đang còn lơ mơ. Hãy nhìn sơ đồ từng bước từ lúc Seller bấm nút "Mua gói" cho tới khi hoàn tất hoặc hoàn tiền:



Giải thích 4 cơ chế bảo vệ trong luồng trên (Giảng viên hỏi câu nào trả lời đúng câu đó):

1. **Chốt chặn 1: Chống bấm đúp / gửi lặp tại cổng API bằng `Idempotency-Key` + `SHA-256 RequestHash`**

- **File**: và .
- Khi gọi `POST /api/v1/listings/{id}/purchase-package`, Client bắt buộc truyền Header `Idempotency-Key`.
- `ListingsController` băm nội dung `{ listingId, request }` bằng **SHA-256** (`WorkflowService.ComputeRequestHash`, `L61-L65`).
- Trong `WorkflowService.StartPurchaseAsync`:
  - Tìm trong bảng `IdempotencyRequests` theo cặp khóa Unique `(UserId, Key)`.
  - **Nếu đã tồn tại và `RequestHash` giống hệt**: Nghĩa là người dùng bấm đúp nút hoặc mạng lag gửi lại cùng 1 lệnh -> trả về lại kết quả `WorkflowDto` của lần đầu tiên (`L24-L27`), **không tạo giao dịch mới**.
  - **Nếu đã tồn tại nhưng `RequestHash` KHÁC**: Nghĩa là Client lấy lại `Idempotency-Key` cũ nhưng đổi tên gói hoặc số tiền -> ném `InvalidOperationException("The Idempotency-Key has already been used with a different request.")` (`L18-L21`).

2. **Chốt chặn 2: Chống mất sự kiện khi RabbitMQ sập bằng `Transactional Outbox Pattern`**
   - **Tại sao không gọi `_publishEndpoint.Publish(...)` trực tiếp trong `WorkflowService`?** Vì nếu `SaveChangesAsync()` vào SQL Server thành công nhưng ngay tích tắc đó RabbitMQ bị đứt mạng (**Lỗi Dual-Write**), thông điệp sẽ bị mất vĩnh viễn và giao dịch của khách bị treo ở trạng thái `Started`.
   - **Cách giải quyết** (): Ghi cả 3 bản ghi: `WorkflowInstance`, `IdempotencyRequest`, và `OutboxMessage("PurchasePackageRequested", payload, correlationId)` vào SQL Server rồi gọi **duy nhất 1 lệnh `await unitOfWork.SaveChangesAsync()`**.
   - Tiến trình nền chạy vòng lặp mỗi **5 giây**, gọi lấy tối đa **50 tin nhắn** có `ProcessedOn == null`, đẩy lên RabbitMQ qua . Gửi thành công thì gán `message.MarkProcessed()` (`ProcessedOn = UtcNow`), nếu lỗi mạng thì tăng `RetryCount++` và ghi `Error` để 5 giây sau thử lại!

3. **Chốt chặn 3: Chống xử lý trùng thông điệp từ RabbitMQ bằng `Inbox Deduplication` + Double Check**
   - RabbitMQ hoạt động theo cơ chế **At-least-once delivery** (có thể phát lại cùng 1 message nếu mạng chập chờn trước khi Consumer kịp ACK).
   - **Lớp bảo vệ 1 (Tầng Filter hạ tầng)**: được gắn vào toàn bộ Consumer trong . Trước khi cho message vào Consumer (`await next.Send(context)`), nó kiểm tra bảng `InboxMessages` xem cặp khóa Unique `(ConsumerName, MessageId)` đã tồn tại chưa (`L54-L65`). Nếu có rồi -> `return;` bỏ qua ngay. Sau khi Consumer chạy xong -> lưu `(ConsumerName, MessageId)` vào `InboxMessages`.
   - **Lớp bảo vệ 2 (Tầng nghiệp vụ trong từng Consumer)**:

        - kiểm tra bảng `WalletTransactions` xem `CorrelationId` đó đã có giao dịch `Charge` chưa.
        - kiểm tra bảng `ListingHistories` xem đã có bản ghi `UpgradePackage` chứa `CorrelationId` đó chưa.
        - kiểm tra bảng `WalletTransactions` xem `CorrelationId` đó đã có giao dịch `Refund` chưa.

4. **Chốt chặn 4: Giao dịch bù trừ hoàn tiền tự động (Compensating Transaction) trong `PurchasePackageSaga`**
    - **File**:
    - Bảng lưu trạng thái của Saga là `PurchasePackageSagaStates` (cấu hình khóa lạc quan `builder.Property(x => x.Version).IsConcurrencyToken()` trong).
    - Đồng thời, mỗi khi Saga chuyển bước, hàm `RecordStepAsync` (`L160-L191`) cập nhật song song bảng `WorkflowInstances` và thêm dòng mới vào bảng `WorkflowSteps` để API `GET /api/v1/workflows/{correlationId}` theo dõi được tiến độ.
    - **Cách test/chứng minh Hoàn tiền tự động trước mặt Giảng viên trong 30 giây**: Chỉ cần gọi `POST /api/v1/listings/{id}/purchase-package` với một mã gói không tồn tại, ví dụ `"packageCode": "VIP_ERROR"` và `"chargeAmount": 150000`.
        - Bước 1: `DeductWalletConsumer` chỉ cần biết `UserId` và `Amount = 150000`, nên **trừ tiền ví thành công** và bắn `WalletDeducted`.
        - Bước 2: `UpgradeListingConsumer` tìm gói `"VIP_ERROR"` trong bảng `PackageDefinitions` -> không thấy (`package is null`) -> bắn `ListingUpgradeFailed` !
        - Bước 3: `PurchasePackageSaga` đang ở `UpgradingListing` nhận được `ListingUpgradeFailed` (`L105-L121`) -> chuyển sang `Compensating` và bắn lệnh `RefundWalletCommand`.
        - Bước 4: `RefundWalletConsumer` cộng trả lại `150000` vào `MainBalance` của ví, ghi `WalletTransaction(Refund)` và bắn `WalletRefunded` -> Saga chuyển sang `Compensated` !

Luồng 5: Tác vụ nền Quartz.NET (`ListingExpirationJob`) & Kiểm duyệt tự động (`AmeModerationConsumer`)

1. **Quartz.NET quét tin hết hạn (&)**:
    - Được cấu hình trong với biểu thức Cron `"0 0/5 * * * ?"` (chạy định kỳ mỗi 5 phút).

- Có gắn `[DisallowConcurrentExecution]` trên đầu class `ListingExpirationJob` (`L8`) để nếu lần chạy trước chưa xong thì lần chạy sau không được đè lên.
- Xử lý theo lô (**Batch Size = 100 bản ghi/lần**, `L12` của `ListingExpirationService.cs`): Tìm các tin có `Status == Published && EndDate <= now`, gọi `listing.Expire(now)` chuyển sang `Expired`, ghi `ListingHistory("Expire")` và ghi `OutboxMessage("ListingExpired")`.

2. **Luồng AME Moderation ()**:
   - Khi Seller nhấn Submit tin đăng (`ListingService.SubmitAsync`), hệ thống ghi `OutboxMessage("ListingSubmitted")`.
   - `OutboxDispatcherWorker` đẩy sự kiện `ListingSubmitted` lên RabbitMQ -> `AmeModerationConsumer` bắt sự kiện này, nạp danh sách luật `ContentRules` và `MarketPriceBenchmarks` từ DB rồi gọi sang dịch vụ Python FastAPI (`IAmeModerationClient`).
   - **Điểm cực kỳ an toàn**: Nếu bạn không bật server Python FastAPI khi đi thi, khối `try-catch` tại `L129-L137` của `AmeModerationConsumer.cs` sẽ tự động bắt lỗi kết nối và **Fallback về** `ManualReview` (`listing.MarkForManualReview()`), giữ nguyên trạng thái `PendingModeration` để Moderator/Admin duyệt tay trên giao diện!

PHẦN 3: LUYỆN TẬP 6 ĐỀ THI "THỰC HÀNH TẠI CHỖ" (PHẦN A — 45 PHÚT)

Nhìn vào **Đề số 7** trong ảnh bạn gửi, cấu trúc đề thi Phần A của thầy luôn yêu cầu **sửa/mở rộng một tính năng có sẵn trên mã nguồn của nhóm** và kiểm tra bằng dữ liệu đầu vào cụ thể. Dưới đây là **6 đề thi sát nhất** với mã nguồn PropNest kèm **công thức sửa code từng file**:

ĐỀ DỰ ĐOÁN SỐ 1 (Giống hệt Đề số 7 trong ảnh): Mở rộng API Tìm kiếm & Lọc Tin đăng (`GET /api/v1/listings`)

Đề bài giả định:

> Mở rộng API tìm kiếm tin đăng công khai (`GET /api/v1/listings`) để đáp ứng thêm các tiêu chí sau:
>
> 1. Khi tìm theo từ khóa `q` , tìm trong `Title` , `Address` **HOẶC** `Description` (không phân biệt hoa thường).
> 2. Bổ sung lọc theo diện tích tối thiểu (`minArea`) và diện tích tối đa (`maxArea`).
> 3. Sắp xếp các tin đăng có gói `"VIP"` lên đầu danh sách, sau đó mới sắp xếp theo `CreatedAt` giảm dần.

Cách làm trong 5 phút (Chỉ sửa đúng 2 file):

1. **Bước 1**: Mở , thêm `MinArea` và `MaxArea` vào `GetPublicListingsQuery` :

```
csharp
public sealed record GetPublicListingsQuery(
    string? Q = null,
    string? City = null,
    string? District = null,
    string? Ward = null,
    string? PropertyType = null,
    string? ListingType = null,
    string? Type = null,
    decimal? MinPrice = null,
    decimal? MaxPrice = null,
    decimal? MinArea = null, // <-- Thêm dòng này
    decimal? MaxArea = null, // <-- Thêm dòng này
    int? Page = null,
    int PageNumber = 1,
    int PageSize = 10);
```

2. **Bước 2**: Mở tại hàm `GetPublicListingsAsync` :

```
csharp
var keyword = query.Q?.Trim();
var queryable = dbContext.Listings.AsNoTracking()
    .Where(x => x.Status == ListingStatus.Published)
    // 1. Tìm trong Title, Address HOẶC Description:
    .Where(x => string.IsNullOrWhiteSpace(keyword)
             || x.Title.Contains(keyword)
             || x.Address.Contains(keyword)
             || (x.Description != null && x.Description.Contains(keyword)))
    .Where(x => query.District == null || x.District == query.District)
    .Where(x => query.City == null || x.City == query.City)
    .Where(x => query.Ward == null || x.Ward == query.Ward)
    .Where(x => query.PropertyType == null || x.PropertyType == query.PropertyType)
    .Where(x => targetType == null || x.ListingType == targetType)
    .Where(x => query.MinPrice == null || x.Price >= query.MinPrice)
    .Where(x => query.MaxPrice == null || x.Price <= query.MaxPrice)
    // 2. Lọc theo MinArea và MaxArea:
    .Where(x => query.MinArea == null || x.Area >= query.MinArea)
    .Where(x => query.MaxArea == null || x.Area <= query.MaxArea);

var totalCount = await queryable.CountAsync(cancellationToken);

var items = await queryable
    // 3. Ưu tiên gói VIP lên đầu, sau đó mới xếp theo CreatedAt giảm dần:
    .OrderByDescending(x => x.PackageCode == "VIP")
    .ThenByDescending(x => x.CreatedAt)
    .Skip((query.PageNumber - 1) * query.PageSize)
    .Take(query.PageSize)
    .Select(x => new ListingSummaryDto(...))
    .ToListAsync(cancellationToken);
```

(Lưu ý: Collation mặc định của SQL Server là `CI` - Case Insensitive nên `.Contains(keyword)` trong EF Core dịch ra `LIKE N'%keyword%'` đã tự động không phân biệt hoa thường!)

ĐỀ DỰ ĐOÁN SỐ 2: Mở rộng API Lịch sử Giao dịch Ví (`GET /api/v1/wallets/me/transactions`)

Đề bài giả định:

> Mở rộng API lấy danh sách giao dịch ví (`GET /api/v1/wallets/me/transactions`):
>
> 1. Cho phép lọc thêm theo khoảng số tiền (`minAmount` , `maxAmount`) và tìm theo từ khóa trong `description` (`keyword`).
> 2. Chỉ trả về các giao dịch của chính user đang đăng nhập, sắp xếp mới nhất lên trước.

Cách làm trong 5 phút (Sửa 3 file):

1. Mở : Thêm tham số `[FromQuery] decimal? minAmount, [FromQuery] decimal? maxAmount, [FromQuery] string? keyword` vào `GetMyTransactions` .
2. Mở và : Truyền tiếp các tham số xuống `IWalletRepository` .
3. Mở : Thêm `.Where(t => minAmount == null || t.Amount >= minAmount)` , `.Where(t => maxAmount == null || t.Amount <= maxAmount)` , `.Where(t => string.IsNullOrWhiteSpace(keyword) || t.Description.Contains(keyword))` .

ĐỀ DỰ ĐOÁN SỐ 3: Thêm/Sửa Quy tắc Nghiệp vụ trong Domain (`Listing` hoặc `Wallet`)

Đề bài giả định:

> Sửa quy tắc nghiệp vụ:
>
> 1. Khi nạp tiền vào ví (`POST /api/v1/wallets/top-up`), nếu số tiền nạp từ `1,000,000` VNĐ trở lên thì tự động thưởng thêm **10%** vào ngăn khuyến mãi (`PromoBalance`) và ghi thêm 1 bản ghi `WalletTransaction` loại `PromoReward` .
> 2. Hoặc: Không cho phép Seller gửi duyệt (`Submit`) tin đăng nếu tin đăng chưa có ít nhất 1 tấm ảnh hoặc mô tả ngắn hơn 20 ký tự.

Cách làm trong 5 phút:

- **Trường hợp 1 (Thưởng 10% vào `PromoBalance` khi nạp >= 1 triệu)**:
    1. Mở : Thêm hàm `CreditPromo(decimal amount)` :

```
csharp
public void CreditPromo(decimal amount)
{
    EnsurePositive(amount);
    PromoBalance += amount;
    Touch();
}
```

2. Mở trong hàm `TopUpAsync`:

```
csharp
if (command.Amount >= 1_000_000m)
{
    var bonus = command.Amount * 0.10m;
    var promoBefore = wallet.MainBalance + wallet.PromoBalance;
    wallet.CreditPromo(bonus);
    var promoAfter = wallet.MainBalance + wallet.PromoBalance;
    var bonusTx = new WalletTransaction(
        walletId: wallet.WalletId,
        correlationId: command.CorrelationId ?? Guid.NewGuid(),
        type: WalletTransactionType.PromoReward,
        amount: bonus,
        balanceBefore: promoBefore,
        balanceAfter: promoAfter,
        description: "Thưởng 10% khuyến mãi nạp tiền");
    await walletRepository.AddTransactionAsync(bonusTx, cancellationToken);
}
```

- **Trường hợp 2 (Kiểm tra điều kiện khi `Submit` tin đăng)**: Mở tại hàm `SubmitForModeration()` (hoặc):

```
csharp
if (string.IsNullOrWhiteSpace(Description) || Description.Length < 20)
{
    throw new InvalidOperationException("Mô tả tin đăng phải có ít nhất 20 ký tự trước khi gửi duyệt.");
}
```

ĐỀ DỰ ĐOÁN SỐ 4 (Lab 07 - Task 1): Viết Custom Middleware mới & Chứng minh vị trí đặt trong Pipeline

Đề bài giả định:

> Viết một Middleware `RequestTimingMiddleware` đo thời gian xử lý mỗi HTTP Request bằng mili-giây, đính kèm vào Response Header `X-Execution-Time-Ms`, hoặc chặn các request gửi lên nếu có Header `X-Client-Blocked: true`.

Cách làm trong 3 phút:

1. Tạo file `src/PropNest.Api/Middleware/RequestTimingMiddleware.cs` (sử dụng `Stopwatch.GetTimestamp()` và `context.Response.OnStarting(...)` như ở **Lab 07 - Mã nguồn 5**).
2. Đăng ký vào **ngay sau** `CorrelationIdMiddleware` và **trước** `UseAuthentication()`:

```
csharp
app.UseMiddleware<ExceptionHandlingMiddleware>();
app.UseMiddleware<CorrelationIdMiddleware>();
app.UseMiddleware<RequestTimingMiddleware>(); // Đặt ở đây để log có sẵn CorrelationId và đo được cả thời gian Auth + Controller
```

ĐỀ DỰ ĐOÁN SỐ 5 (Lab 07 - Task 2): Thêm Policy Authorization / Phân quyền mới

Đề bài giả định:

> Hiện tại cả `Moderator` và `Admin` đều dùng chung Policy `CanModerateListings`. Hãy tách riêng một Policy mới tên là `AdminOnly` (hoặc yêu cầu email phải có đuôi `@propnest.vn`) và áp dụng riêng cho API xem thống kê doanh thu `GET /api/v1/moderation/stats` (Moderator chỉ được duyệt tin, không được xem doanh thu ví).

Cách làm trong 2 phút:

1. Mở :

```
csharp
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("CanModerateListings", policy => policy.RequireRole("Moderator",
"Admin"));
    options.AddPolicy("AdminOnly", policy => policy.RequireRole("Admin"));
});
```

2. Mở , gắn `[Authorize(Policy = "AdminOnly")]` lên hàm `GetDashboardStats`:

```
csharp
[HttpGet("dashboard-stats")]
[HttpGet("stats")]
[Authorize(Policy = "AdminOnly")]
public async Task<IActionResult> GetDashboardStats(CancellationToken
cancellationToken)
```

ĐỀ DỰ ĐOÁN SỐ 6 (Lab 07 - Task 3): Thêm trường mới vào Entity `Listing` + EF Core Migration + Cập nhật `DeltaEngine`

Đề bài giả định:

> Thêm trường `Bedrooms` (Số phòng ngủ, kiểu `int?`, không được âm) vào thực thể `Listing`. Cập nhật API tạo/sửa/xem tin đăng, cập nhật `DeltaEngine` để lưu vết khi sửa `Bedrooms`, và chạy Migration cập nhật CSDL.

Checklist 6 vị trí cần chạm tới (Làm đúng thứ tự này sẽ không bao giờ sót hoặc lỗi biên dịch):

1. **Domain Entity**: -> Thêm `public int? Bedrooms { get; private set; }`, thêm tham số `int? bedrooms = null` vào Constructor, `UpdateContent` và `ApplyContent` (kiểm tra `if (bedrooms < 0) throw new ArgumentOutOfRangeException(...)`).
2. **EF Core Config**: -> (Kiểu `int?` EF Core tự map thành cột `int NULL`, không bắt buộc cấu hình thêm hoặc thêm `builder.Property(x => x.Bedrooms);`).
3. **Contracts & DTOs**:
   - -> Thêm `int? Bedrooms = null` vào `ListingDto`, `CreateListingCommand`, `UpdateListingCommand`.
   - -> Thêm `int? Bedrooms = null` vào `CreateListingRequest`, `UpdateListingRequest`.

4. **Application Service & DeltaEngine**:
    - -> Truyền `command.Bedrooms` vào `new Listing(...)`, `listing.UpdateContent(...)` và `ToDto(...)`.
    - -> Thêm dòng `AddDelta(deltas, nameof(ListingDto.Bedrooms), before.Bedrooms, after.Bedrooms);`.
5. **Controller**: -> Truyền `request.Bedrooms` trong `Create` và `Update`.
6. **Chạy lệnh Migration**:

```
powershell
dotnet ef migrations add AddBedroomsToListing --project src/PropNest.Infrastructure --
startup-project src/PropNest.Api
dotnet ef database update --project src/PropNest.Infrastructure --startup-project
src/PropNest.Api
```

PHẦN 4: NHỮNG ĐIỂM LỆCH NHỎ GIỮA BÁO CÁO VÀ CODE THỰC TẾ CẦN LƯU Ý TRƯỚC NGÀY THI

Trong thông báo thi có nhắc: "*xử lý dứt điểm các chỗ lệch cấu hình/lỗi build*". Khi đối chiếu 100% giữa file báo cáo PDF và mã nguồn hiện tại trên máy bạn, có **4 điểm quan trọng** bạn cần nắm rõ để không bị bất ngờ:

1. **Điểm 1 — Mapping mã lỗi `InvalidOperationException` giữa Middleware và Controller**:
    - Trong : `InvalidOperationException` trả về **`409 Conflict`** (đúng như Bảng 1 của Lab 03 trong báo cáo).
    - Nhưng trong : Dòng 44 hiện đang viết `InvalidOperationException => StatusCodes.Status400BadRequest`.
    - **Hệ quả**: Khi test `Idempotency-Key` trùng key nhưng khác payload tại `POST /api/v1/listings/{id}/purchase-package` (hàm này không dùng `this.HandleException` mà rơi ra `ExceptionHandlingMiddleware`), nếu dòng 44 của `ExceptionHandlingMiddleware.cs` để là `Status400BadRequest` thì API sẽ trả về `400` thay vì `409 Conflict` như trong báo cáo (Lab 06 & Chương 8)!
    - **Khuyến nghị**: Sửa dòng 44 của thành `StatusCodes.Status409Conflict` để khớp 100% với báo cáo!
2. **Điểm 2 — Kích hoạt Serilog trong `PropNest.Api/Program.cs`**:

- File đã được viết sẵn hàm `builder.ConfigureSerilog()`, và bên đã gọi `builder.ConfigureSerilog();`.
- Tuy nhiên trong hiện chưa có dòng `builder.ConfigureSerilog();`. Nếu giảng viên kiểm tra file log JSON `logs/propnest-YYYYMMDD.json` bên phía `PropNest.Api`, bạn chỉ cần thêm `using PropNest.Api.Logging;` và `builder.ConfigureSerilog();` ngay dưới dòng `var builder = WebApplication.CreateBuilder(args);`.

3. **Điểm 3 — Thuộc tính `[Authorize]` trên `WalletsController`**:
   - Trong , class hiện tại chỉ gắn `[Route("api/v1/wallets")]` và `[ApiController]`, chưa gắn `[Authorize]` ở đầu class (dù bên trong hàm vẫn kiểm tra `currentUser.UserId ?? throw new UnauthorizedAccessException(...)` -> trả về `403 Forbidden`). Nếu thêm `[Authorize]` lên đầu `WalletsController`, khi khách chưa đăng nhập gọi vào sẽ trả về chuẩn `401 Unauthorized` ngay từ tầng Middleware.
4. **Điểm 4 — Thứ tự khởi chạy hệ thống trước khi bước vào phòng thi**: Trước giờ thi 15 phút, luôn bật sẵn theo thứ tự sau và kiểm tra không có bất kỳ lỗi nào:

```
powershell
# 1. Bật container RabbitMQ (đảm bảo cổng 5672 và 15672 đang Up/healthy)
docker compose -f deploy/docker-compose.yml up -d

# 2. Kiểm tra build toàn bộ Solution (0 Warning, 0 Error) và chạy 20 Unit Tests
dotnet build PropNest.slnx
dotnet test tests/PropNest.Domain.Tests/PropNest.Domain.Tests.csproj

# 3. Chạy API (Terminal 1) và Worker (Terminal 2)
dotnet run --project src/PropNest.Api
dotnet run --project src/PropNest.Workflow.Worker
```